МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА

Институт радиоэлектроники и информационных технологий

«Визуализация и спектральный анализ с Python»

ОТЧЕТ

по лабораторной работе № 2

по дисциплине

Программирование на Python

РУКОВОДИТЕЛЬ:

_____ Зуев А. Б.

(подпись) (Фамилия, И., О.)

СТУДЕНТ:

_____ Акифьев И.И.

(подпись) (Фамилия, И., О.)

24-ИСТ-2

(шифр группы)

Работа защищена «___» _____

С оценкой _____

Нижний Новгород

2026

Содержание

ОТЧЕТ 1
Цели и задачи 3
Описание проделанной работы 4
Вывод 7
Листинг программы 7

Цели и задачи

Цель работы: визуализация и спектральный анализ с Python.

Задачи:

Записать WAV-файл с собственной речью (фраза с именем и фамилией) с частотой дискретизации 4000 Гц, форматом PCM signed 16 bit, моно. Длительность записи — не более 3 секунд. Имя файла — «1.wav».

Разработать программу на Python, которая реализует:

Чтение из WAV-файла дискретных отсчетов звукового сигнала и визуализацию первых N отсчетов (N вводится с клавиатуры) в виде линейного сплошного графика (согласно варианту №1).

Построение осциллограммы всего сигнала как функции времени.

Спектральный анализ всех отсчетов сигнала с вычислением модуля ДПФ ($\sqrt{Re^2 + Im^2}$) и построение графика спектра в зависимости от частоты.

Построение **гистограммы** распределения амплитуд отсчетов.

Оформить все графики с подписями осей, указанием размерности и заголовками.

Обеспечить корректную обработку ошибок (отсутствие файла, неверный ввод) и использовать регулярные выражения для проверки ввода.

Измерить и вывести время выполнения программы.

Описание проделанной работы

Подготовка исходных данных

Согласно варианту №1, была выбрана частота дискретизации 4000 Гц. В программе Audacity был создан и записан WAV-файл с фразой, содержащей имя и фамилию. Параметры файла:

Частота дискретизации: 4000 Гц

Каналы: моно (1 канал)

Разрядность: 16 bit PCM signed

Длительность: 2,430 секунды

Имя файла: 1.wav

Структура программы

Программа написана на языке Python с использованием следующих библиотек:

Библиотека	Назначение
numpy	Работа с массивами данных, математические операции (БПФ, sqrt, статистика)
matplotlib.pyplot	Построение всех графиков
scipy.io.wavfile	Чтение WAV-файлов
re	Проверка пользовательского ввода с помощью регулярных выражений
time	Измерение времени выполнения программы

Описание ключевых функций и блоков программы

Для чтения файла используется функция wavfile.read(), которая возвращает частоту дискретизации и массив отсчетов. Для корректной обработки ошибок используется конструкция try-except, перехватывающая исключения FileNotFoundError и любые другие возможные ошибки при чтении файла.

```python
filename = "1.wav"

try:
    # Чтение wav-файла
    # frequency - частота дискретизации (Гц)
    # data - массив отсчетов (для mono - одномерный)
    frequency, data = wavfile.read(filename)

    print(f"Файл: {filename}")
    print(f"Частота дискретизации: {frequency} Гц")
    print(f"Форма данных: {data.shape}")
    print(f"Тип данных: {data.dtype}")

    # Проверяем, что частота соответствует варианту
    if frequency != 4000:
        print(f"Внимание: частота в файле {frequency} Гц, а должна быть 4000 Гц")

except FileNotFoundError:
    print(f"Ошибка: файл '{filename}' не найден!")
    print("Создайте файл с записью вашей речи (имя+фамилия) с частотой 4000 Гц, моно, 16 bit")
    exit(1)
except Exception as e:
    print(f"Ошибка при чтении файла: {e}")
    exit(1)

# Нормализуем данные для удобства (переводим в диапазон [-1, 1])
# Для int16 диапазон значений от -32768 до 32767
data_normalized = data / 32768.0
```

Рисунок 1. Блок чтения WAV-файла и нормализации данных.

Для ввода количества отсчетов, отображаемых на первом графике, используется цикл while True и регулярное выражение r'^\d+$', которое проверяет, что пользователь ввел только цифры. После успешной проверки введенное значение преобразуется в целое число и проверяется на попадание в допустимый диапазон (от 1 до общего количества отсчетов в файле).

```python
# Используем регулярное выражение для проверки ввода
# \d+ означает одну или более цифр
pattern = re.compile(r'^\d+$')

while True:
    user_input = input(f"Введите количество отсчетов для визуализации (1-{len(data)}): ")
    if pattern.match(user_input):
        n_samples = int(user_input)
        if 1 <= n_samples <= len(data):
            break
        else:
            print(f"Число должно быть от 1 до {len(data)}")
    else:
        print("Пожалуйста, введите целое положительное число")

print(f"Будет отображено {n_samples} отсчетов")
```

Рисунок 2. Блок ввода и проверки данных с использованием регулярных выражений

Визуализация первых N отсчетов (График №1)

Согласно варианту №1, тип графика — линейный сплошной. Для построения используется функция plt.plot() с параметром 'b-' (синяя сплошная линия). По оси X откладываются номера отсчетов, по оси Y — нормализованные амплитуды.

Рисунок 3. Визуализация первых N отсчетов (линейный сплошной график)

Построение осциллограммы (График №2)

Осциллограмма строится как зависимость амплитуды от времени. Массив времени вычисляется по формуле: time = номер_отсчета /

частота_дискретизации. График выполняется зеленой линией ('g-'). На график также выводится информационная рамка с длительностью сигнала и частотой дискретизации.

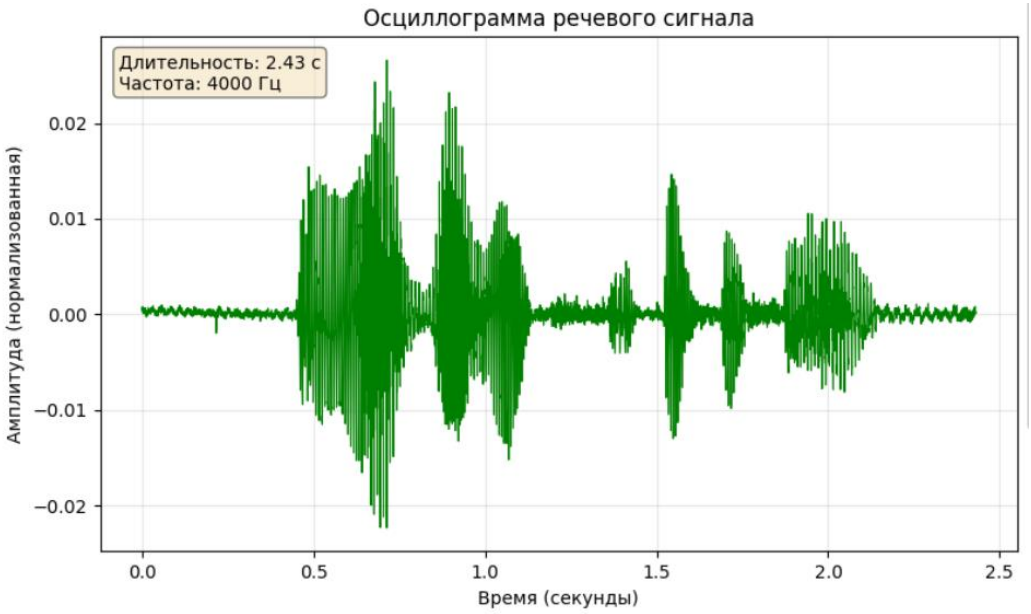

Рисунок 4. Осциллограмма звукового сигнала.

Спектральный анализ (График №3)

Для спектрального анализа используется быстрое преобразование Фурье (БПФ) из библиотеки NumPy: np.fft.fft(). Согласно варианту №1, требуется вычислить **модуль ДПФ**: $|X(k)| = \sqrt{Re^2 + Im^2}$. После вычисления спектра частоты получаются с помощью np.fft.fftfreq(), после чего отбрасываются отрицательные частоты (так как для реального сигнала они симметричны). График спектра выполняется красной линией ('r-'). Ось X ограничена частотой Найквиста (frequency/2 = 2000 Гц).

Рисунок 5. Спектральный анализ звукового сигнала(модуль ДПФ)

Построение гистограммы (График №4)

Гистограмма амплитуд строится с помощью функции `plt.hist()` с параметрами:

- `bins=50` — разбиение диапазона на 50 интервалов;
- `density=True` — нормировка (показывается относительная частота);
- `color='purple'` — фиолетовый цвет столбцов;
- `edgecolor='black'` — черные границы столбцов.

На гистограмму выводится информационная рамка со средним значением и стандартным отклонением.

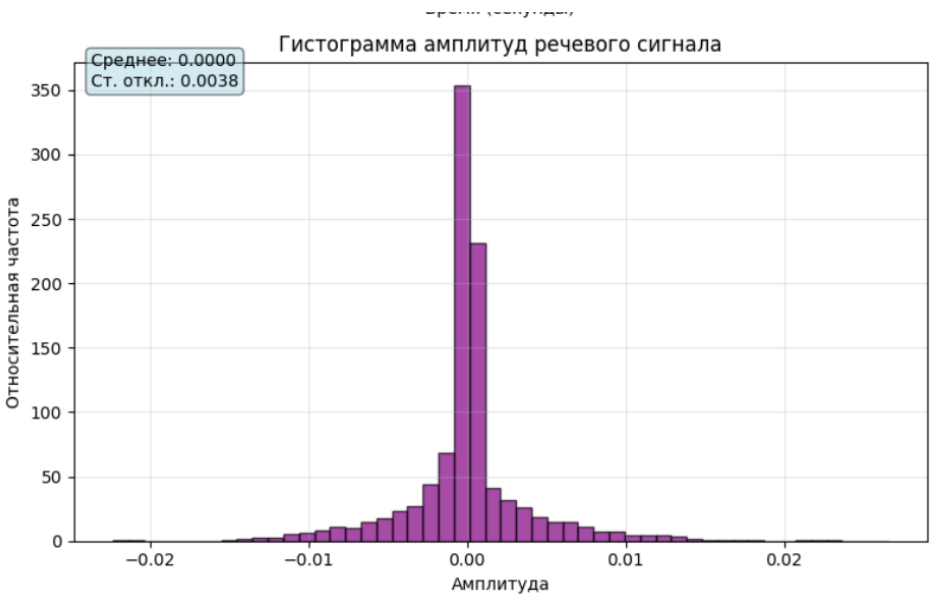

Рисунок 7. Гистограмма амплитуд

Пример работы программы (консольный вывод)

```
Файл: 1.wav
Частота дискретизации: 4000 Гц
Форма данных: (9720,)
Тип данных: int16
Введите количество отсчетов для визуализации (1-9720): 9700
Будет отображено 9700 отсчетов

Время выполнения программы: 11.0959 секунд

--- Информация о сигнале ---
Всего отсчетов: 9720
Длительность: 2.430 секунд
Минимальная амплитуда: -0.0223
Максимальная амплитуда: 0.0266
Средняя амплитуда: 0.0000
Энергия сигнала (приблизительно): 0.1398

Process finished with exit code 0
```

Рисунок 8. Консольный вывод программы

Вывод

В результате выполнения лабораторной работы мною были решены следующие задачи: написание программы на языке программирования Python, которая загружает звуковой файл, читает его и на основе полученных данных визуализирует дискретные отсчеты звукового сигнала в виде линейного сплошного графика (согласно варианту №1), строит осциллограмму, спектральный анализ (модуль ДПФ) и гистограмму распределения амплитуд.

Листинг программы

```
"""
Лабораторная работа №2
Вариант 1: Линейный сплошной график + Модуль ДПФ
Частота дискретизации: 4000 Гц
"""

import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
import re
```

```
import time

# Засекаем время начала выполнения программы
start_time = time.time()

# ==================== 1. ЧТЕНИЕ WAV-ФАЙЛА ====================

filename = "1.wav"

try:
    # Чтение wav-файла
    # frequency - частота дискретизации (Гц)
    # data - массив отсчетов (для mono - одномерный)
    frequency, data = wavfile.read(filename)

    print(f"Файл: {filename}")
    print(f"Частота дискретизации: {frequency} Гц")
    print(f"Форма данных: {data.shape}")
    print(f"Тип данных: {data.dtype}")

    # Проверяем, что частота соответствует варианту
    if frequency != 4000:
        print(f"Внимание: частота в файле {frequency} Гц, а должна быть 4000 Гц")

except FileNotFoundError:
    print(f"Ошибка: файл '{filename}' не найден!")
    print("Создайте файл с записью вашей речи (имя+фамилия) с частотой 4000 Гц, моно, 16
bit")
    exit(1)
except Exception as e:
    print(f"Ошибка при чтении файла: {e}")
    exit(1)

# Нормализуем данные (переводим в диапазон [-1, 1])
# Для int16 диапазон значений от -32768 до 32767
data_normalized = data / 32768.0

# ==================== 2. ВВОД КОЛИЧЕСТВА ОТСЧЕТОВ С ПРОВЕРКОЙ
====================

# Используем регулярное выражение для проверки ввода
# \d+ означает одну или более цифр
pattern = re.compile(r'^\d+$')

while True:
    user_input = input(f"Введите количество отсчетов для визуализации (1-{len(data)}): ")
    if pattern.match(user_input):
        n_samples = int(user_input)
        if 1 <= n_samples <= len(data):
            break
        else:
            print(f"Число должно быть от 1 до {len(data)}")
```

```
    else:
        print("Пожалуйста, введите целое положительное число")

print(f"Будет отображено {n_samples} отсчетов")

# ==================== 3. ГРАФИК №1: Визуализация первых N отсчетов
====================
# Тип графика: Линейный сплошной график

plt.figure(figsize=(16, 10))

# Подграфик 1: Визуализация первых N отсчетов
plt.subplot(2, 2, 1)
# Создаем массив номеров отсчетов от 0 до n_samples-1
samples_indices = np.arange(n_samples)
# Линейный сплошной график
plt.plot(samples_indices, data_normalized[:n_samples], 'b-', linewidth=1)
plt.title(f'Визуализация первых {n_samples} отсчетов сигнала')
plt.xlabel('Номер отсчета (n)')
plt.ylabel('Амплитуда (нормализованная)')
plt.grid(True, alpha=0.3)
plt.xlim(0, n_samples - 1)

# ==================== 4. ГРАФИК №2: Осциллограмма ====================
# Сигнал как функция времени

# Создаем массив времени (в секундах)
# Время для k-го отсчета = k / частота_дискретизации
time_axis = np.arange(len(data)) / frequency

plt.subplot(2, 2, 2)
plt.plot(time_axis, data_normalized, 'g-', linewidth=0.8)
plt.title('Осциллограмма речевого сигнала')
plt.xlabel('Время (секунды)')
plt.ylabel('Амплитуда (нормализованная)')
plt.grid(True, alpha=0.3)
# Добавляем информацию о длительности
duration = len(data) / frequency
plt.text(0.02, 0.9, f'Длительность: {duration:.2f} с\nЧастота: {frequency} Гц',
        transform=plt.gca().transAxes, bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

# ==================== 5. СПЕКТРАЛЬНЫЙ АНАЛИЗ ====================
# Модуль ДПФ: sqrt(Re² + Im²)
# Используем БПФ (быстрое преобразование Фурье) более быстрая реализация, результат
тот же

# Выполняем ДПФ (БПФ) для всего сигнала
fft_result = np.fft.fft(data_normalized)

# Re² + Im² (квадрат модуля)
magnitude_squared = np.real(fft_result) ** 2 + np.imag(fft_result) ** 2
```

```
# Модуль ДПФ = sqrt(Re² + Im²)
magnitude = np.sqrt(magnitude_squared)

# Создаем массив частот (Гц)
# fftfreq возвращает частоты для каждой точки ДПФ
#f(k) = k / (n × d) = k × frequency / n Где n = len(data)
frequencies = np.fft.fftfreq(len(data), d=1 / frequency)

# Нас интересуют только положительные частоты (от 0 до частоты Найквиста), так как
сигнал симметричен
# Частота Найквиста = frequency/2 = 2000 Гц
positive_freq_idx = frequencies >= 0
frequencies_pos = frequencies[positive_freq_idx]
magnitude_pos = magnitude[positive_freq_idx]

plt.subplot(2, 2, 3)
plt.plot(frequencies_pos, magnitude_pos, 'r-', linewidth=0.8)
plt.title('Спектр сигнала (Модуль ДПФ)')
plt.xlabel('Частота (Герцы, Гц)')
plt.ylabel('Амплитуда спектра |X(f)|')
plt.grid(True, alpha=0.3)
plt.xlim(0, frequency / 2)  # До частоты Найквиста
plt.axvline(x=100, color='gray', linestyle='--', alpha=0.5, label='Форманты обычно до 3-4 кГц')
plt.legend()

# ==================== 6. ГРАФИК №4: Гистограмма ====================
# Гистограмма отсчетов - показывает распределение амплитуд

plt.subplot(2, 2, 4)
# hist - строит гистограмму
# bins=50 - разбиваем диапазон амплитуд на 50 интервалов
# density=True - нормируем (площадь = 1), показываем частоту, а не количество
plt.hist(data_normalized, bins=50, color='purple', alpha=0.7, density=True, edgecolor='black')
plt.title('Гистограмма амплитуд речевого сигнала')
plt.xlabel('Амплитуда')
plt.ylabel('Относительная частота')
plt.grid(True, alpha=0.3)
# Добавляем информацию о среднем и стандартном отклонении
mean_val = np.mean(data_normalized)
std_val = np.std(data_normalized)
plt.text(0.02, 0.95, f'Среднее: {mean_val:.4f}\nСт. откл.: {std_val:.4f}',
        transform=plt.gca().transAxes, bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.5))

# ==================== 7. НАСТРОЙКА ОБЩЕГО ВИДА ====================

plt.suptitle(f'Анализ речевого сигнала (Вариант 1, файл: {filename})', fontsize=14,
fontweight='bold')
plt.tight_layout()  # Автоматически подгоняет расположение графиков
plt.show()

# ==================== 8. ДОПОЛНИТЕЛЬНАЯ ИНФОРМАЦИЯ
====================
```

```
execution_time = time.time() - start_time
print(f"\nВремя выполнения программы: {execution_time:.4f} секунд")
print("\n--- Информация о сигнале ---")
print(f"Всего отсчетов: {len(data)}")
print(f"Длительность: {len(data) / frequency:.3f} секунд")
print(f"Минимальная амплитуда: {np.min(data_normalized):.4f}")
print(f"Максимальная амплитуда: {np.max(data_normalized):.4f}")
print(f"Средняя амплитуда: {np.mean(data_normalized):.4f}")
print(f"Энергия сигнала (приблизительно): {np.sum(data_normalized ** 2):.4f}")
```

Ссылка на Гитхаб репозиторий: https://github.com/f1a71in3/python_lab_2

Процент использования ИИ в коде – 70%. Deepseek
ИИ использовался для обучения новым функциям и библиотекам, а так же помогал выстраивать логику программы.